

TECHNICAL DESIGN

# Design and Implementation Guide

Architecture, code organization, data flow, operations, and maintenance

|            |                                                     |
|------------|-----------------------------------------------------|
| System     | NSNS Waiver Application                             |
| Status     | As implemented                                      |
| Technology | ASP.NET Core Razor Pages / .NET 10 / MySQL / Dapper |
| Prepared   | 2026-07-30                                          |

**Audience:** Developers and operators maintaining the ASP.NET Core Razor Pages application, MySQL schema, email outbox, administrator area, and Railway deployment.

## Document contents

- 1. Design goals and constraints
- 2. Runtime architecture
- 3. Project structure and components
- 4. Core request workflows
- 5. Data design and migrations
- 6. Security and validation design
- 7. Email outbox design
- 8. Administrator design
- 9. Configuration and deployment
- 10. Testing and maintenance
- 11. Function reference

## 1. Design goals and constraints

- Keep one ASP.NET Core Razor Pages project; do not introduce MVC controllers, Blazor, a SPA framework, or Entity Framework.
- Use dependency injection and async/await throughout request and persistence paths.
- Keep UI concerns in PageModels, business rules in services, and SQL in repositories.
- Use Dapper with MySqlConnection and parameterized SQL only.
- Store every timestamp in UTC and generate the signed time on the server.
- Preserve the agreement as a deployed HTML file with no database versioning or snapshots.
- Commit a waiver and its related family/email records atomically.

## 2. Runtime architecture

The application is a layered monolith. Razor Pages receive HTTP requests and map untrusted input into service requests. Services enforce business rules and construct domain records. Repositories execute parameterized Dapper SQL. MySQL stores submissions and the reliable email outbox. A hosted worker delivers queued messages through SMTP.

| Layer                                                                                                                                                                                                                                               | Primary responsibility                                                               | Key implementation                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------|
| Presentation                                                                                                                                                                                                                                        | Public form, confirmation, error/privacy pages, admin login and list.                | Pages/*.cshtml and PageModel classes     |
| Business                                                                                                                                                                                                                                            | Event resolution, validation, normalization, record construction, email composition. | WaiverSubmissionService                  |
| Persistence                                                                                                                                                                                                                                         | Transactional writes and read projections.                                           | Repositories/*Repository.cs              |
| Infrastructure                                                                                                                                                                                                                                      | MySQL connections, SMTP, background polling, configuration.                          | Data, SmtpEmailSender, EmailOutboxWorker |
| Domain/data contracts                                                                                                                                                                                                                               | Submission, family member, outbox, admin projection, requests/results.               | Models and Services records              |
| <b>Dependency direction:</b> Pages depend on service/repository abstractions. Services depend on repository abstractions. Repositories depend on the database connection factory. Infrastructure implementations are wired centrally in Program.cs. |                                                                                      |                                          |

## 3. Project structure and components

| Path       | Responsibility                                                                                        |
|------------|-------------------------------------------------------------------------------------------------------|
| Program.cs | Registers configuration, authentication, repositories, services, SMTP, hosted worker, and middleware. |

| Path              | Responsibility                                                                                    |
|-------------------|---------------------------------------------------------------------------------------------------|
| Pages/            | Razor markup and PageModels for public and administrator workflows.                               |
| Services/         | Business validation, agreement loading, credential checking, SMTP sending, and outbox processing. |
| Repositories/     | Dapper SQL and transactional persistence.                                                         |
| Models/           | Database-backed domain records and admin list projection.                                         |
| Options/          | Strongly typed Waiver, Email/SMTP, and Admin configuration.                                       |
| Data/             | MySQL connection factory and primary migration copies.                                            |
| Database/Scripts/ | Operational copies of schema and upgrade SQL.                                                     |
| Content/          | Deployable waiver-agreement.html.                                                                 |
| Tests/            | Unit, PageModel, SQL, processor, credential, and optional MySQL integration tests.                |
| wwwroot/          | Application CSS/JavaScript and vendored Bootstrap/jQuery assets.                                  |

## 4. Core request workflows

### 4.1 Display the waiver form

1. IndexModel.OnGetAsync receives the event query-string value.
2. WaiverSubmissionService.FindEvent normalizes and resolves it against Waiver:Events.
3. FileWaiverAgreementProvider.GetAsync reads Content/waiver-agreement.html.
4. The PageModel renders the event, agreement, form fields, repeatable family controls, required waiver checkbox, and default-selected optional media checkbox.
5. If the event is unknown, the page shows an event-not-found state. If the agreement is unavailable/unapproved, submission controls are disabled.

### 4.2 Submit a waiver

6. Razor model binding and DataAnnotations perform initial validation and anti-forgery validation occurs automatically.
7. IndexModel.OnPostAsync re-resolves the event, reloads the agreement, verifies required agreement acceptance, and maps a SubmitWaiverRequest.
8. WaiverSubmissionService.SubmitAsync trims, validates, normalizes, and accumulates all field errors.
9. The service creates a server UTC timestamp, UUID reference, WaiverSubmission, family records, and two HTML-safe outbox messages.
10. WaiverSubmissionRepository.CreateSubmissionAsync opens a transaction and inserts the parent, children, and outbox rows.

11. After commit, the PageModel places safe confirmation values in TempData and redirects to /Confirmation.
12. ConfirmationModel.OnGet consumes TempData and redirects stale/direct visits back to the index.

### 4.3 Review submissions

13. LoginModel validates configured credentials and creates the cookie principal.
14. SubmissionsModel parses allow-listed sort values and clamps the page number.
15. AdminSubmissionRepository selects from the 200 newest records and aggregates family members with GROUP\_CONCAT.
16. The Razor table displays 20 rows per page and uses links to toggle sort direction.
17. Logout deletes the cookie and returns to the login page.

## 5. Data design and migrations

### 5.1 Tables

| Table                 | Purpose                                                  | Key relationships/indexes                                                                             |
|-----------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| waiver_submissions    | Parent waiver and signer record.                         | Unique submission_reference; indexes on event, normalized contact data, event/email, and signed time. |
| waiver_family_members | Zero-to-many family participants.                        | Foreign key to submission with ON DELETE CASCADE; submission_id index.                                |
| email_outbox          | Durable customer and owner messages plus delivery state. | Foreign key to submission with ON DELETE CASCADE; status/retry and creation indexes.                  |

### 5.2 Transaction boundary

CreateSubmissionAsync inserts the submission first to obtain its numeric ID, then assigns that ID to every family member and outbox message. A single MySQLTransaction covers every insert. Any exception triggers rollback with CancellationToken.None so cancellation cannot leave a partial waiver.

### 5.3 Migration strategy

- 001\_create\_waiver\_tables.sql creates the three permitted tables for a new database.
- 002\_add\_media\_release\_agreed.sql upgrades an existing database with a NOT NULL boolean after agreed.
- DEFAULT TRUE initializes historical submissions as agreed and supports inserts that omit the new column.

- Normal application inserts always pass `MediaReleaseAgreed` explicitly; the database default does not override a posted opt-out.
- Apply migration 002 once before deploying application code that selects or inserts `media_release_agreed`.

## 6. Security and validation design

| Control                    | Implementation                                                                                                                     |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Server validation          | Both <code>PageModel</code> and <code>WaiverSubmissionService</code> validate; service validation remains authoritative.           |
| SQL injection prevention   | All values use Dapper parameters. Admin sort columns map from an enum to fixed SQL fragments.                                      |
| Cross-site request forgery | Razor Pages form posts use the framework anti-forgery token.                                                                       |
| Output encoding            | Razor encodes normal output; email bodies use <code>HtmlEncoder</code> for customer-supplied values.                               |
| Authentication             | Cookie authentication protects <code>/Admin</code> ; cookie is HTTP-only, <code>SameSite Strict</code> , sliding, and eight hours. |
| Credential comparison      | Credentials are SHA-256 hashed before fixed-time byte comparison.                                                                  |
| Open redirect prevention   | Login accepts <code>ReturnUrl</code> only when <code>Url.IsLocalUrl</code> returns true.                                           |
| Secret handling            | Configuration supplies database, admin, and SMTP secrets; code avoids logging those values.                                        |
| Request metadata           | IP and user agent are length-limited; user agent is truncated to 500 characters before mapping.                                    |

## 7. Email outbox design

Email is separated from the HTTP transaction. Submission success depends on queuing the messages, not on contacting SMTP. This prevents a temporary mail outage from losing or delaying the customer waiver.

18. `EmailOutboxWorker` starts only when `Email:Enabled` is true.
19. The worker creates a dependency-injection scope and calls `ProcessBatchAsync`.
20. `GetPendingAsync` selects Pending rows and Failed rows whose next attempt time has arrived.
21. `SmtplibEmailSender` validates settings and sends an HTML `MailMessage`.
22. Success changes status to Sent and clears `last_error`.
23. Failure stores a safe error type/summary and schedules exponential backoff of 1, 2, 4, 8, 16, 32, then at most 64 minutes.
24. When the next attempt reaches `MaximumAttempts`, `next_attempt_at_utc` becomes null and the repository records Abandoned.

**Operational behavior:** When no messages are eligible, the worker waits `PollIntervalSeconds`. Unexpected processor errors are logged and followed by the same delay. Application shutdown

cancellation exits cleanly.

## 8. Administrator design

### 8.1 Authentication

Admin credentials are configured, not stored in MySQL. The login page is anonymous; the submissions page is authorized. Successful sign-in creates a name claim under the cookie scheme.

### 8.2 Query and pagination

The repository first limits the dataset to the 200 newest submissions, then applies the selected sort and a 20-row LIMIT/OFFSET. This guarantees a maximum of ten pages and prevents older records from entering the admin browsing window merely because a different sort is selected.

### 8.3 Display projection

AdminSubmissionListItem contains only fields needed by the table. A correlated GROUP\_CONCAT subquery formats family names and optional relationships into one display string.

## 9. Configuration and deployment

| Configuration                | Purpose                                                |
|------------------------------|--------------------------------------------------------|
| ConnectionStrings:Default    | MySQL connection used by all repositories.             |
| Waiver:BusinessOwnerEmail    | Recipient of every owner notification.                 |
| Waiver:Events                | Allowed event-code to event-name dictionary.           |
| Email:Enabled                | Starts or disables the background delivery worker.     |
| Email:FromAddress / FromName | Sender identity shown on outgoing messages.            |
| Email:PollIntervalSeconds    | Idle/error polling delay, clamped to 1-300 seconds.    |
| Email:BatchSize              | Messages processed per batch, clamped to 1-100.        |
| Email:MaximumAttempts        | Delivery attempts before abandonment, clamped to 1-20. |
| Email:Smtp:*                 | SMTP host, port, TLS flag, username, and password.     |
| Admin:Username / Password    | Credentials for the protected administrator area.      |

### 9.1 Deployment sequence

25. Provision MySQL and restrict network/database credentials appropriately.

26. Apply 001 for a new database, or apply outstanding numbered migrations in order.
27. Provide connection string, events, business-owner email, admin credentials, and SMTP settings through Railway variables or secure configuration.
28. Build the Docker image and deploy the ASP.NET Core application.
29. Verify the public event URL, one submission, both outbox rows, email delivery, admin login, and admin display.
30. Monitor application logs for outbox failures without logging recipient PII or secrets.

## 10. Testing and maintenance

### 10.1 Automated coverage

- WaiverSubmissionService tests cover normalization, validation, duplicate acceptance, HTML encoding, media opt-out, emails, and owner configuration.
- Index PageModel tests cover event/agreement loading, form mapping, required agreement, and confirmation redirect.
- Repository SQL and migration tests verify explicit parameter use, required tables, keys, indexes, and media column.
- Email processor tests cover successful delivery, retry scheduling, cancellation, and abandonment.
- Credential and admin/page tests cover authentication and display behavior.
- MySQL integration tests verify commit, rollback, generated IDs, relationships, and UTC mapping when WAIVERAPP\_TEST\_MYSQL\_CONNECTION is configured.

### 10.2 Routine maintenance checklist

- Before code changes, review PROJECT.md and AGENTS.md.
- Keep agreement edits limited to Content/waiver-agreement.html and obtain business/legal approval.
- Add numbered migrations for schema changes; never assume CREATE TABLE IF NOT EXISTS alters an existing table.
- Update model, request, service, repository SQL, emails/admin projection, and tests together when adding a submission field.
- Run dotnet build and the full test suite; configure MySQL integration tests for release-critical database changes.
- Confirm email HTML in common clients when changing markup or inline styles.
- Do not log secrets, connection strings, passwords, or unnecessary customer data.

### 10.3 Known maintenance considerations

**Agreement quality:** FileWaiverAgreementProvider considers any nonblank file approved unless it

contains REPLACE\_WITH\_APPROVED\_WAIVER. Legal approval and text-encoding quality remain operational responsibilities.

- System.Net.Mail SMTP behavior should be revalidated if hosting or TLS requirements change.
- The simple configured admin credential model is suitable for the current small protected area; multiple users, password rotation, audit trails, or MFA would require a new authentication design.
- The outbox query does not claim/lock rows for multiple worker instances. Scale-out deployment would require row claiming or distributed coordination to prevent duplicate sends.
- Media consent defaults to selected and historical migration values default to true; changes to consent policy require coordinated UI, schema, wording, and legal/privacy review.

## 11. Function reference

| Component / function                             | What it does                                                                                      |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------|
| IndexModel.OnGetAsync                            | Resolves the event and loads agreement content for the public form.                               |
| IndexModel.OnPostAsync                           | Validates page state, maps input, submits, stores TempData, and redirects.                        |
| WaiverSubmissionService.FindEvent                | Normalizes a code and resolves it from configured allowed events.                                 |
| WaiverSubmissionService.SubmitAsync              | Performs authoritative validation, constructs all records, and invokes transactional persistence. |
| ValidateFamilyMembers                            | Enforces the ten-member limit and validates each nested entry.                                    |
| CreateOutboxMessages                             | Creates the customer confirmation and owner notification.                                         |
| CreateBossNotificationBody                       | Builds HTML-safe details including event highlight and media choice.                              |
| WaiverSubmissionRepository.CreateSubmissionAsync | Commits submission, members, and emails in one MySQL transaction.                                 |
| EmailOutboxRepository.GetPendingAsync            | Returns pending and due-for-retry messages in creation order.                                     |
| EmailOutboxProcessor.ProcessBatchAsync           | Sends a batch and records sent, retry, or abandoned state.                                        |
| EmailOutboxWorker.ExecuteAsync                   | Runs the background polling loop and creates a scope per batch.                                   |
| SmtpEmailSender.SendAsync                        | Validates SMTP configuration and sends one HTML message.                                          |
| AdminCredentialValidator.IsValid                 | Checks both configured credentials using fixed-time comparisons.                                  |
| LoginModel.OnPostAsync                           | Authenticates the administrator and safely redirects.                                             |
| SubmissionsModel.OnGetAsync                      | Parses list controls, counts records, clamps pagination, and loads one page.                      |
| AdminSubmissionRepository.GetRecentAsync         | Queries a safe sort column and aggregates family display data.                                    |
| FileWaiverAgreementProvider.GetAsync             | Reads the HTML agreement and reports whether submissions may proceed.                             |
| ConfirmationModel.OnGet                          | Consumes one-time confirmation data and blocks direct/stale access.                               |
| MySQLConnectionFactory.OpenConnectionAsync       | Creates an open UTC-aware MySQL connection and cleans up on failure.                              |